

Indian Legal AI Helper

A Context-Aware Legal Question-Answering System
Grounded in Indian Law

Devansh Verma (220346)

Kushal Bansal (220575)

Alankrit Gupta (220110)

Karunaya Garg (220507)

Kushal Gupta (220576)

CS787: Introduction to Generative AI
November 23, 2025

Abstract

This report presents the Indian Legal AI Helper, an advanced Retrieval-Augmented Generation (RAG) system designed to make Indian legal information accessible through accurate, cited, and context-aware responses. The system processes over 170,000 legal document chunks from diverse sources including the Constitution of India, Supreme Court judgments, IPC, CrPC, and legal Q&A datasets. Using sentence-transformers for embedding, FAISS for vector search, and Google Gemini 2.5 Flash for generation, our system achieved 93.99% faithfulness, 85.15% answer relevancy, and maintained a hallucination rate of only 1.01% across 100 test questions. The deployed application is publicly accessible at <https://legal-ai-app-njbxx5tsbhunjjeib9lre.streamlit.app/>.

1 Introduction

1.1 Problem Context

Accessing and understanding Indian legal information remains a major challenge for the general public and professionals without formal legal training. Existing resources suffer from:

- Dense legal terminology (legalese) that creates barriers to understanding
- Vast, unstructured, and poorly indexed law repositories spanning 75+ years
- Lack of contextual explanation and user-friendly summaries
- Fragmented information across multiple sources and formats

These barriers significantly impede legal awareness, timely decision-making, and access to justice for common citizens.

1.2 Proposed Solution

We developed the Indian Legal AI Helper—a production-ready RAG system that provides:

- **Curated Legal Corpus:** 170,163 processed chunks from Constitution of India, major Acts (IPC, CrPC), 26,688 Supreme Court judgments (1950-2024), and legal Q&A datasets
- **Contextual Q&A:** AI pipeline that retrieves relevant provisions and generates concise, accessible explanations

- **Source Attribution:** Every answer includes explicit citations with document references
- **User-Centric Design:** Clean Streamlit interface optimized for non-lawyers
- **Live Deployment:** Publicly accessible web application

2 System Architecture

2.1 Technology Stack

Component	Technology
Web Framework	Streamlit
Embedding Model	sentence-transformers/all-MiniLM-L6-v2 (384-dim)
Vector Database	FAISS (Facebook AI Similarity Search)
Generation LLM	Google Gemini 2.5 Flash
Evaluation LLM	Cohere Command-R-08-2024
Core Libraries	google-generativeai, faiss-cpu, numpy
Language	Python 3.10+
Deployment	Streamlit Community Cloud

Table 1: Technology Stack

2.2 RAG Pipeline Architecture

Our system implements a standard RAG pipeline with the following stages:

1. **Data Ingestion:** Collected diverse Indian legal data from:
 - Hugging Face: ILC Dataset (3,073 cases)
 - Kaggle: Supreme Court Judgments (26,688 PDFs, 1950-2024)
 - IndicLegalQA: 10,000 Q&A pairs from SC judgments
 - Indian Legal QA: 14,543 Q&A pairs (Constitution, IPC, CrPC)
 - Case Summarization Dataset: 14,887 judgment documents
2. **Data Processing:** All text data cleaned and segmented into overlapping chunks (500 chars with 50-char overlap) to optimize retrieval precision
3. **Embedding & Indexing:** Each chunk converted to 384-dimensional vectors using all-MiniLM-L6-v2 and stored in FAISS L2 index for rapid similarity search
4. **Query-Time Processing:**
 - User query embedded using same model
 - FAISS index searched for top-5 semantically similar chunks
 - Retrieved chunks compiled into structured prompt with metadata
 - Prompt sent to Gemini 2.5 Flash for answer generation
 - Citations extracted and confidence score calculated

2.3 Dual-LLM Architecture

We employed a dual-LLM strategy for optimal performance:

- **Google Gemini 2.5 Flash:** Production deployment in Streamlit app
 - Fast response times suitable for real-time user interaction
 - Strong instruction-following for citation requirements
 - Deployed via google-generativeai API
- **Cohere Command-R-08-2024:** Evaluation and development testing
 - Specialized RAG capabilities for benchmark evaluation
 - Used to avoid Gemini API rate limits during intensive testing
 - JSON response formatting for automated metric extraction

3 Data Collection & Processing

3.1 Dataset Summary

Dataset	Source	Documents
ILC Cases	Hugging Face	3,073
IndicLegalQA	Mendeley	10,000
Indian Legal QA	Kaggle	14,543
SC Judgments	Kaggle	26,688
Case Summarization	Kaggle	14,887
Total Documents		69,191
Total Processed Chunks		170,163

Table 2: Final Dataset Statistics

3.2 Data Processing Pipeline

Text Extraction:

- PDFs processed using PyPDF2 and pdfplumber
- JSON datasets loaded and validated
- All text normalized to UTF-8 encoding

Chunking Strategy:

- Sentence-based chunking with 500-character target size
- 50-character overlap between chunks to preserve context
- Legal boundary preservation (sections, articles maintained intact)
- Metadata attached: {title, source, section}

Embedding Generation:

- Batch processing with size 64 for efficiency
- 384-dimensional dense vectors via all-MiniLM-L6-v2
- Total processing time: ~4.8 minutes on GPU

4 Evaluation Methodology

4.1 Test Dataset Construction

We created a comprehensive ground truth dataset of 100 questions:

- **50 CLAT Questions:** Extracted from CLAT 2022 & 2023 legal reasoning passages covering Constitutional Law, Criminal Procedure, Contract Law, Evidence Act, Corporate Law, and International Law
- **50 Constitution Questions:** Direct knowledge questions about Indian Constitutional provisions, procedures, and institutions

Each question includes:

- Original question text
- Ground truth answer (gold standard)
- Source passage (for CLAT questions)
- Topic classification

4.2 Evaluation Metrics

We employed a comprehensive evaluation framework across three dimensions:

4.2.1 Generative Accuracy

- **Semantic Similarity:** Cosine similarity between generated and ground truth answer embeddings (normalized 0-1)
- **F1 Score:** Token-level overlap measuring precision and recall
- **Answer Relevancy:** LLM-judged relevance to user question (0-1 scale)

4.2.2 Trust & Grounding

- **Faithfulness:** LLM-judged measure of whether answer derives only from retrieved context (0-1 scale)
- **Hallucination Rate:** Percentage of answers with faithfulness < 0.5

4.2.3 Retrieval Quality

- **System Confidence:** Calculated as $\min(2 \times \text{avg_similarity}, 1.0)$ where $\text{similarity} = \frac{1}{1 + \text{L2_distance}}$
- **Citation Presence:** Binary indicator of whether sources were cited

4.3 LLM-as-Judge Approach

For faithfulness and relevancy, we used Cohere Command-R as an automated judge with the following prompt structure:

You are an expert legal AI evaluator. Grade the AI Answer based on User Question and Context.

Provide two scores (0.0 to 1.0):

1. "faithfulness": Is answer derived **only** from Context?
2. "relevancy": Does answer address User Question?

Return ONLY JSON: {"faithfulness": 0.9, "relevancy": 0.8}

5 Results

5.1 Final Performance Report

Category	Metric	Score
3* Generative Accuracy	Semantic Similarity	72.98%
	F1 Score	14.86%
	Answer Relevancy	85.15%
2* Trust & Grounding	Faithfulness	93.99%
	Hallucination Rate	1.01%
Retrieval Quality	System Confidence	99.85%

Table 3: Final Evaluation Results (99/100 questions successfully processed)

5.2 Key Findings

Strengths:

- **Excellent Faithfulness (93.99%):** System rarely hallucinates, maintaining strong grounding in retrieved sources
- **Low Hallucination Rate (1.01%):** Only 1 out of 99 successfully processed questions contained unsupported claims
- **High Relevancy (85.15%):** Generated answers effectively address user questions
- **Strong Retrieval (99.85% confidence):** FAISS retrieval consistently finds relevant legal documents

Observations:

- **Low F1 Score (14.86%):** Expected due to paraphrasing—legal answers are rephrased from sources rather than copied verbatim, which is actually desirable for accessibility
- **Moderate Semantic Similarity (72.98%):** Reflects intentional reformulation into plain language while preserving legal accuracy

5.3 Sample Interaction

Question: “What is the punishment for theft under Indian law?”

Generated Answer: “The punishment for theft under Indian law is imprisonment for up to 3 years, with or without fine [SOURCE 5]. However, the provided context mentions ‘Imprisonment for 10 years and fine,’ which appears to be a discrepancy...”

Confidence: 100.0%

Citations:

- Source 1: CRPC Q&A - Indian Legal QA
- Faithfulness: 0.95
- Relevancy: 0.90

6 Implementation Details

6.1 Streamlit Application

The production application (`app.py`) features:

- **Caching:** `@st.cache_resource` for one-time model loading
- **Error Handling:** Graceful degradation with informative error messages
- **Citation Display:** Expandable sections showing source documents
- **Confidence Indicator:** Visual feedback on answer reliability
- **Source Transparency:** Full retrieved document preview available

6.2 Deployment Configuration

```
# requirements.txt
streamlit
sentence-transformers
faiss-cpu
google-generativeai
numpy

# .streamlit/secrets.toml (not in repo)
GOOGLE_API_KEY = "your-key-here"
```

Large File Storage:

- FAISS index (1.9 GB) and chunks JSON (130 MB) stored via Git LFS
- Automatically downloaded during deployment

6.3 Evaluation Pipeline

Key optimizations implemented:

- **Batch Processing:** Pause 60s after every 5 questions to respect Cohere’s 10 calls/minute limit
- **Retry Logic:** Exponential backoff for rate limit errors (429) and server issues (503)

- **Skip-on-Fail:** Gracefully skip questions where generation/judging fails rather than blocking pipeline
- **Combined Judging:** Single LLM call evaluates both faithfulness and relevancy simultaneously

Total evaluation time: ~27.5 minutes for 100 questions.

7 Challenges & Solutions

7.1 API Rate Limiting

Challenge: Cohere Trial key limited to 10 calls/minute; Gemini free tier has quota constraints.

Solution: Implemented dual-LLM architecture—Gemini for production, Cohere for evaluation with batch processing and exponential backoff retry logic.

7.2 Data Scale

Challenge: Processing 26,688 Supreme Court PDFs would require excessive time and storage.

Solution: Prioritized datasets 1, 2, 3, 5 (structured text) for knowledge base; reserved Dataset 4 for future expansion.

7.3 Low F1 Scores

Challenge: Token-level F1 scores were initially concerning (14.86%).

Solution: Recognized this reflects desirable paraphrasing behavior. Added semantic similarity and LLM-judged metrics to better assess answer quality.

7.4 Citation Extraction

Challenge: Automatically detecting which sources were actually used in generated answers.

Solution: Implemented pattern matching for [SOURCE N] tags in LLM responses and cross-referenced with retrieved documents.

8 Deployment & Accessibility

8.1 Live Application

The system is publicly accessible at:

<https://legal-ai-app-njbxk5tsbhunjjeib9lre.streamlit.app/>

8.2 Repository Structure

```
legal-ai-app/
  app.py                # Streamlit application
  AI_Legal_Rag.ipynb    # Development notebook
  requirements.txt       # Python dependencies
  README.md             # Documentation
  images/
    ss.png              # Application screenshot
  ground_truth_100.json # Evaluation dataset
  main_chunks.json      # Knowledge base (Git LFS)
  main_chunk_metadata.json # Chunk metadata
  main_legal_index.faiss # FAISS index (Git LFS)
```

9 Conclusion

We successfully developed and deployed a production-ready Legal RAG system that achieves:

- **High Faithfulness (93.99%):** Strong grounding in retrieved legal sources
- **Minimal Hallucinations (1.01%):** Reliable and trustworthy responses
- **Good Relevancy (85.15%):** Answers effectively address user questions
- **Public Accessibility:** Live deployment with clean UI for non-lawyers

The system demonstrates that RAG architectures can effectively democratize access to complex legal information while maintaining high accuracy and transparency through explicit source citations.

9.1 Future Work

- **Multi-turn Conversations:** Add conversation history for follow-up questions
- **Legal Domain Fine-tuning:** Fine-tune embedding model on legal corpus
- **Human Evaluation:** Conduct user studies with legal professionals and laypeople
- **Multilingual Expansion:** Add support of multiple languages to help users in every medium

Acknowledgments

We thank the course instructors of CS787: Introduction to Generative AI for their guidance. We acknowledge Hugging Face, Kaggle, and Mendeley for providing open legal datasets. API access was provided by Google (Gemini) and Cohere.

References

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
2. ILC Dataset: d0r1h/ILC on Hugging Face
3. IndicLegalQA: Legal Question Answering Dataset (Mendeley)
4. Supreme Court Judgments India 1950-2024 (Kaggle)
5. Indian Legal QA Dataset (Kaggle: akshatgupta7)